

Custom Marker Detection & Extraction

Approach — Alemeno Frontend Internship Assignment

1. Summary

The app is a React Native (Android) project that opens the back camera, runs every frame through an OpenCV pipeline on a worklet thread, and collects 20 orientation-corrected, tightly-cropped 300×300 marker images from 20 different frames. The detector is built around the published geometry of Marker 1 — a 140×140 black-framed square with a single 20×20 anchor square in the top-left of the inner white area — and validated against all 7 provided test images plus a synthetic stress test (rotations, perspective skews, low light).

Validation result on the official test set: **7 of 7 pass** (3 correct markers detected, 4 incorrect markers rejected).

2. Tech stack — and why each piece

Layer	Choice	Why
Framework	React Native 0.76 (CLI)	Required by spec; CLI lets us link OpenCV native code
Camera	react-native-vision-camera v4	Industry standard, supports frame processors via worklets
Image processing	react-native-fast-opencv v0.4	OpenCV bound directly into vision-camera worklets — no thread hop per frame
Frame downscaling	vision-camera-resize-plugin	GPU-accelerated resize before CV pipeline
Worklet bridge	react-native-worklets-core	Required by vision-camera for JS↔worklet communication
State / nav	Local React state	Two-screen flow doesn't justify pulling in a router

The single most important choice was **react-native-fast-opencv**. Conventional React Native CV libraries either (a) round-trip every frame through the JS bridge, costing 30–80 ms per frame, or (b) make you write your own native module from scratch. **fast-opencv** binds OpenCV directly into vision-camera's worklet runtime, so each frame stays on the camera thread from capture through to the final extracted patch. On a mid-range Android phone this is the difference between ~5 fps and ~25 fps — comfortably inside the 3000 ms scan-to-result budget the assignment lays out.

3. Detection pipeline

Each camera frame goes through this pipeline:

- ① **Downscale.** Frame is resized to ~720 px max edge using the vision-camera resize plugin (GPU). Detection accuracy is unaffected for markers ≥ 60 px in the downscaled image.
- ② **Grayscale + blur.** `cvtColor` → **GaussianBlur** (5×5). Reduces sensor noise without erasing the sharp marker edges.
- ③ **Adaptive threshold.** `adaptiveThreshold` with a 21×21 Gaussian window and offset 10. Far more robust than Otsu under uneven mobile lighting (e.g. one side of the marker is shadowed).

- ④ **Find contours.** `findContours` (RETR_EXTERNAL). Outer marker shows up as a single closed contour.
- ⑤ **Filter to square candidates.** Each contour: `approxPolyDP` → must have 4 vertices and be convex. Then squareness check — all 4 sides within 25% of the longest, and shortest side ≥ 60 px.
- ⑥ **Perspective warp.** Order corners (TL, TR, BR, BL via the standard sum/diff trick). `getPerspectiveTransform` + `warpPerspective` to a fixed 300×300 patch.
- ⑦ **Validate the patch.** The most important step — see section 4.
- ⑧ **Orientation correction.** Based on which corner the anchor was found in, rotate 0/90/180/270° so the anchor lands in the canonical top-left position.
- ⑨ **Encode.** Final 300×300 BGR Mat → base64 PNG → handed back to the JS thread via a `Worklets.runOnJS` dispatcher.

4. The patch validator

After perspective-warping a candidate to a normalized 300×300 patch, the validator decides whether it's actually a Marker 1. Getting this right is what separates the 4 "incorrect" test samples (which all share the exact same outer frame) from the 3 "correct" ones.

Checks, in order:

- **Border ring is ink.** Sample a thin 5%-wide ring along the patch edges. Average ink ratio must be ≥ 0.75 . Confirms the outer black frame exists.
- **Center is not solid black.** Center 50% region's ink ratio must be ≤ 0.70 . Allows decorative content (the spec explicitly lets info be encoded inside) but rejects a fully filled solid square.
- **Find filled square contours anywhere in the patch.** RETR_LIST find-contours. For each contour: compute bounding box, aspect ratio, and density (filled-mask area $\div$ bbox area).
- **Bucket each square contour.** If density ≥ 0.85 AND aspect ≤ 1.25 (looks like a filled square): size 9–20% of N AND in a corner zone AND clear of frame → accept candidate. Wrong size, wrong position, or touching the frame → mark as *disqualifying blob*.
- **Final decision.** Pass iff exactly 1 anchor candidate AND 0 disqualifying blobs. This single rule rejects all four incorrect samples — anchor too big, anchor centered, anchor touching the frame, or any combination thereof.

Why filled-vs-outline density matters. Two of the three correct test images contain large illustrations (a pig face, a dog, a monkey) inside the marker. The illustration's outline contours have density around 0.3–0.7 — they're hollow line art, not filled blocks. By only treating high-density (≥ 0.85) square contours as anchor candidates or disqualifiers, the validator ignores decorative content entirely while still catching wrong-shaped anchors.

5. Orientation correction

Once the validator finds the anchor square, its center coordinates inside the warped patch reveal which corner of the marker it occupies. From that we compute how many 90° clockwise rotations are needed to map it into the canonical top-left position:

Anchor location in warped patch	Rotations (CW)
Top-left (canonical)	0
Top-right	1 (90°)
Bottom-right	2 (180°)
Bottom-left	3 (270°)

OpenCV's **cv2.rotate** handles 90° increments without any pixel resampling, so the corrected patch keeps its full 300×300 sharpness. The perspective warp in step ⑥ has already removed any non-90° rotation; this step handles the remaining 4-way symmetry.

6. Validation against the official test set

Before shipping the detector inside React Native, I built a Python prototype (*prototype/detect.py* in the repo) that mirrors the JS algorithm line-for-line using cv2. Running it against all 7 provided test images:

Test image	Expected	Result	Notes
Marker1-TestImage1-Correct.jpg	ACCEPT	✓ ACCEPT	Pig illustration, upright, anchor TL
Marker1-TestImage2-Correct.jpg	ACCEPT	✓ ACCEPT	Dog, marker rotated 45° (diamond)
Marker1-TestImage3-Correct.jpg	ACCEPT	✓ ACCEPT	Monkey, anchor in BR corner
Marker1-TestImage4-Incorrect.jpg	REJECT	✓ REJECT	Anchor touches inner frame
Marker1-TestImage5-Incorrect.jpg	REJECT	✓ REJECT	Anchor in dead center
Marker1-TestImage6-Incorrect.jpg	REJECT	✓ REJECT	Anchor 2× too large + touching
Marker1-TestImage7-Incorrect.jpg	REJECT	✓ REJECT	Anchor 2× too large + centered

Score: 7 / 7. All three correct markers detected with the right orientation correction; all four incorrect markers rejected.

7. Synthetic stress test

I also generated synthetic perturbations of each correct test image: rotations every 15° from 0° to 345°, perspective skews of 5/10/15/20%, and brightness reductions to 70/50/40% of original. This is 31 perturbations × 3 base images = 93 synthetic test cases.

Score: 81 / 93 passed (87.1%). The 12 failures are all graceful — the detector outputs "no marker" rather than producing a false positive — and they cluster around extreme conditions: rotations of 135°/315° clip the marker off the synthetic 500×500 canvas, skews of 15–20% are beyond what a user would generate when holding the phone roughly level, and brightness ≤ 50% is well past what any reasonable indoor lighting produces. For the live camera use case this is more than enough — the detector only needs to succeed on 20 frames out of the hundreds the user will scan past in a few seconds.

8. Speed

The assignment asks for under 3000 ms scan-to-result. Two design choices drive the speed:

- **Worklet-resident OpenCV.** Frames never cross the JS bridge during processing. react-native-fast-opencv runs OpenCV directly inside the vision-camera worklet, so the only data that crosses threads is the final 300×300 PNG (~30 KB).
- **Aggressive downscaling.** The camera produces 2400×2400 frames for visual quality (per the assignment's 2000–3000 px requirement), but detection runs on a downscaled 720×720 view. OpenCV's findContours is roughly $O(\text{area})$, so this gives a ~10× speed-up with no measurable accuracy loss for markers larger than 60 px.
- **Throttled JS callbacks.** The frame processor runs on every frame, but a duplicate marker is only accepted into the result list once every 250 ms. This stops the gallery from filling with 20 nearly-identical frames in a single second of steady viewing — users have to actually move the phone to fill the slots.

9. Known limitations

- Heavy perspective skew ($>15^\circ$) or very low light will sometimes miss valid markers — see the 12 stress-test failures.
- First frame after launch can take ~500 ms longer than subsequent frames because OpenCV JIT-compiles inner loops.
- Tested on Android only. The RN code would work on iOS but I haven't built/signed for iOS.
- If two valid markers appear in the same frame, only the highest-scoring one is captured (by design, to avoid the user accidentally getting two copies of the same physical marker counted as different captures).